

Flujo de datos de SQL a Python



Descripción del proyecto

Durante este proyecto se trabajará con la base de datos **Olist**, un marketplace brasileño real, generando tres dataframes exportados desde SQL, para posteriormente seleccionar uno de ellos y aplicar una limpieza preliminar en SQL.

Luego, el dataframe limpio será procesado y documentado en un notebook de Google Colab, donde se realizará un procesamiento y limpieza final de los datos.

El proyecto está dividido en dos partes:

1. **Extracción desde SQL** — Generación de tres dataframes mediante joins entre tablas específicas. **Limpieza preliminar en SQL** — Normalización, filtrado, estandarización de columnas clave de **UNO** de los dataframes.
2. **Procesamiento en Python** — Procesamiento y limpieza en python de **UNO** de los dataframes.

Se evaluará la claridad del proceso, la justificación de las decisiones de limpieza y la organización del notebook.

La base de datos es OLIST, son **1.550.922 filas**. Al terminar tendrás la base olist con 9 tablas. Trabajamos en MySQL

Comprueba que cargó bien

Antes de empezar, ejecuta esto. Las nueve tablas deben coincidir:

USE olist;

```
SELECT 'categoria_traduccion' t, COUNT(*) filas, 71    esperado FROM
categoria_traduccion

UNION ALL SELECT 'products',    COUNT(*),    32951  FROM products

UNION ALL SELECT 'sellers',    COUNT(*),    3095   FROM sellers

UNION ALL SELECT 'customers',  COUNT(*),    99441  FROM customers

UNION ALL SELECT 'geolocation', COUNT(*),    1000163 FROM geolocation

UNION ALL SELECT 'orders',    COUNT(*),    99441   FROM orders

UNION ALL SELECT 'order_items', COUNT(*),    112650  FROM order_items

UNION ALL SELECT 'order_payments', COUNT(*),    103886  FROM order_payments

UNION ALL SELECT 'order_reviews', COUNT(*),    99224   FROM order_reviews;
```

Y comprueba que los acentos portugueses se cargaron bien:

```
SELECT DISTINCT product_category_name FROM products
```

WHERE product_category_name LIKE '%m_veis%' LIMIT 3; -- debe salir 'moveis_...'

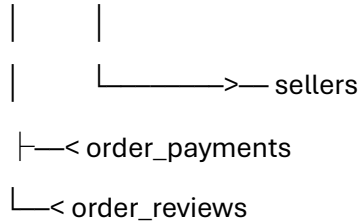
Si algún número no cuadra, vuelve a cargar el volcado antes de seguir.

Note

Son datos **reales** de 99.441 pedidos realizados en Brasil entre 2016 y 2018. Fuente: [Brazilian E-Commerce Public Dataset by Olist](#) · licencia CC BY-NC-SA 4.0.

Modelo de datos

customers —< orders —< order_items >— products >— categoria_traduccion



geolocation se une a customers y sellers por zip_code_prefix

Objetivos concretos

- Comprender la estructura de la base de datos Olist y cómo conectar múltiples tablas.
- Exportar tres dataframes contruidos mediante joins SQL.
- Aplicar reglas de limpieza y estandarización directamente en SQL.
- Crear un notebook ordenado y reproducible que documente los pasos ejecutados.
- Exportar un dataset limpio (CSV o Parquet) para futuras etapas del proyecto.

Dataframes a generar desde SQL

Cada estudiante debe generar **tres** dataframes obligatorios, exportados desde SQL con las tablas indicadas. Posteriormente elegirá uno de ellos para limpiar en SQL y luego en Python.

Para ti:

Dataframe 3: Vendedores y popularidad

Tablas:

- sellers
- order_items
- products

Objetivo: analizar el catálogo por vendedor y la frecuencia con que se vende cada producto, identificando qué vendedores concentran la actividad.

Limpieza preliminar SQL:

- Estandarizar seller_city y seller_state con LOWER(TRIM()). Cuidado: la misma ciudad aparece escrita de varias formas.
- Asegurar que seller_id y product_id existan en sus tablas.
- Filtrar los productos nunca vendidos, o conservarlos y justificarlo.
- Evitar duplicación: order_items tiene una fila **por unidad**, no por pedido. Un pedido de 3 unidades genera 3 filas.

Sugerencias adicionales:

- Tabla agregada con: productos distintos por vendedor, ventas por producto, ingreso total por vendedor.
- Columna derivada ticket_medio por vendedor.